Signaloryx mobile SDKs
Integration and recording guide

25 October 2026 | iOS 0.3.0 release candidate | Android 0.2.0 preview

Release status: iOS 0.3.0 publication is pending successful simulator validation. The published iOS 0.2.0 tag does not contain session timelines or public-label APIs. Do not pin 0.3.0 until that tag is published. This edition replaces older mobile recording instructions.

Use one retained SDK client per project after consent. Native capture is a UIKit/Android View wireframe, not screen video. iOS 0.3.0 joins small uploads into a session timeline and supports explicitly approved static labels. Android 0.2.0 continues to show separate content-free batches. No input values, image pixels or system keyboards are captured.

For the iOS candidate, use the local source ZIP in a staging app until the reviewed SPM tag is published. After publication, pin the exact version below and commit Package.resolved. Rebuilding and releasing your app is required; backend deployment cannot update installed SDK code.

```
.package(
  url: "https://github.com/farazahmed1pk/signaloryx-ios.git",
  exact: "0.3.0" // only after publication
)
```

Reference: signaloryx.com/docs/mobile-integration. iOS source archive: /downloads/signaloryx-ios-source-0.3.0.zip. Android source/AAR and dependency checksums remain in the platform documentation. Verify the published SHA-256 before installation.

Customer privacy: opt in to a small set of reviewed, static public messages. Never approve names, addresses, email, order/payment IDs, input values, arbitrary error bodies or entire customer-detail containers. A blocked ancestor takes precedence over public annotations.

Android: client and consent

Create once after the first consent grant

The following is application integration code. Retain the returned client in your Application or dependency-injection service. Read the consent manager before creating it; a constructed client can restore and drain an existing queue. Use an existing retained client for later grants.

```
import android.content.Context
import com.signaloryx.sdk.Signaloryx
import com.signaloryx.sdk.SignaloryxOptions

fun createTelemetryAfterConsent(
    context: Context,
    consentGranted: Boolean,
    appRelease: String
): Signaloryx {
    check(consentGranted)
    return Signaloryx.create(context.applicationContext,
        SignaloryxOptions(
            dsn = "YOUR_ANDROID_STAGING_DSN",
            baseUrl = "https://signaloryx.com",
            environment = "staging",
            release = appRelease
        )
    )
}
```

Connect consent changes to replay control

The following section defines CappedReplay. Call its methods on the Android main thread. When permission is granted, create the client only if absent, then setEnabled(true) on the retained client. Start only if the Activity is resumed, the workflow is approved, and the sensitive containers have been blocked.

```
// On a consent grant, with a retained client:
telemetry.setEnabled(true)
// If this Activity is resumed and capture is appropriate:
replay.start(consentGranted = true, privateViews = listOf(paymentView))

// On withdrawal, including while the app is foregrounded:
replay.revoke()
// Do not flush or call normal stop before revoking.
// A later grant requires an explicit start again. No auto-restart.
```

Add useful, sanitized investigation context

```
telemetry.setTags(mapOf("flow" to "cart", "market" to "SA"))
telemetry.addBreadcrumb("Cart opened", "navigation")
telemetry.captureLog("Cart update failed",
    level = "error", attributes = mapOf("operation" to "cart.update"))
telemetry.captureException(IllegalStateException("Synthetic cart failure"))
telemetry.captureTransaction("cart.update", 420.0, status = "error")
telemetry.flush() // Delivery attempt, not a receipt confirmation
```

Keep customer data out of messages, tags, attributes and breadcrumbs. Use beforeSend for your reviewed redaction policy before storage/network delivery; it applies to replay batches too. Prefer no user identifier; if required, use an approved opaque ID and clear scope on account change.

Android: lifecycle and duration

This app-owned helper caps each explicit recording start at five foreground minutes. It does not change the native five-frame batch size. Keep one helper for the Activity, call from the main thread, and avoid repeated start calls during recomposition or rendering.

```
import android.app.Activity
import android.os.Handler
import android.os.Looper
import android.view.View
import com.signaloryx.sdk.NativeReplay
import com.signaloryx.sdk.Signaloryx

class CappedReplay(
    private val activity: Activity,
    private val telemetry: Signaloryx
) {
    private val main = Handler(Looper.getMainLooper())
    private var active = false
    private val finish = Runnable { stop() }

    fun start(consentGranted: Boolean, privateViews: List<View>) {
        check(Looper.myLooper() == Looper.getMainLooper())
        if (!consentGranted || active) return
        privateViews.forEach { NativeReplay.block(it) }
        active = telemetry.startReplay(activity,
            consentGranted = true, sampleRate = 1.0)
        if (active) main.postDelayed(finish, 300_000L)
    }

    fun stop() {
        main.removeCallbacks(finish)
        telemetry.stopReplay() // Queues the final partial batch
        active = false
    }

    fun revoke() {
        main.removeCallbacks(finish)
        telemetry.setEnabled(false) // Purges queue and scope
        active = false
    }
}

// In the host Activity; create replay only after the client exists:
override fun onResume() {
    super.onResume()
    // With current consent and an approved screen:
    replay.start(consentGranted, listOf(paymentView))
}
override fun onPause() {
    replay.stop()
    super.onPause()
}
```

The lifecycle snippet assumes the helper exists; when consent has not yet created the client, skip capture. Stop before sensitive flows and Activity destruction. Re-entering/resuming can begin a new five-minute window; persist a deadline in your app if a cap must span Activities or foreground sessions. Use an agreed lower production sampleRate, such as 0.1.

Only the supplied Activity window is traversed. FLAG_SECURE, unfocused windows and invisible views are excluded; WebViews and blocked subtrees are opaque. Separate dialog windows and system keyboards are not captured. Keep FLAG_SECURE protections intact.

iOS 0.3.0: client and consent

Release candidate: simulator validation and publication are pending. The following APIs are absent from published 0.2.0. Retain this coordinator in your app service. Create the client only after consent because it may restore a saved queue. The continuation methods on the next page belong inside this class.

```
import UIKit
import Signaloryx

@MainActor
final class ReplayCoordinator {
    private var telemetry: Signaloryx?
    private var recorder: NativeReplay?
    private weak var activeWindow: UIWindow?

    func begin(in window: UIWindow, consentGranted: Bool,
               privateViews: [UIView]) throws {
        guard consentGranted else { revoke(); return }
        if telemetry == nil {
            var options = try SignaloryxOptions(
                dsn: "YOUR_IOS_STAGING_DSN",
                baseURL: URL(string: "https://signaloryx.com")!)
            options.environment = "staging"
            options.release = Bundle.main.infoDictionary?[
                "CFBundleShortVersionString"] as? String
            telemetry = try Signaloryx(options: options)
        }
        guard let telemetry = telemetry else { return }
        telemetry.setEnabled(true)
        if let previous = activeWindow, previous !== window {
            telemetry.stopReplay()
        }
        activeWindow = window
        recorder = telemetry.startReplay(in: window,
            consentGranted: true, sampleRate: 1.0) // staging QA
        privateViews.forEach { recorder?.block($0) }
    }
    // Add the continuation methods here, inside this class.
}
```

Start after the approved scene becomes active. Repeated valid starts for the same window preserve the recorder and sampling decision. Apply private blocks before the first one-second sample. Set an agreed production sample rate in your app; 1.0 is for staging QA. Only the supplied approved key window is captured.

iOS 0.3.0: public labels and popup messages

Add these methods inside ReplayCoordinator. StaticString restricts this example to literal public messages. The SDK accepts String; only pass reviewed templates or approved localized constants, never actual view text or arbitrary server errors.

```
func publicLabel(_ label: UILabel, message: StaticString) {
    recorder?.allowPublicText(label,
        text: String(describing: message))
}
func publicButton(_ button: UIButton, message: StaticString) {
    recorder?.allowPublicText(button,
        text: String(describing: message))
}
func makePrivateAgain(_ view: UIView) {
    recorder?.maskText(view)
}
func endConsentedSession() {
    telemetry?.stopReplay() // queues a partial consented batch
    recorder = nil; activeWindow = nil
}
func revoke() {
    telemetry?.setEnabled(false) // discard frames AND queue first
    recorder = nil; activeWindow = nil
}
```

Annotate an app-owned UIKit popup before capture

```
// Retained coordinator + actual UIKit views owned by your app:
try replay.begin(in: window, consentGranted: consentAllowed,
    privateViews: [customerDetailsContainer, paymentContainer])
replay.publicLabel(cartTitleLabel, message: "Your shopping bag")
replay.publicButton(retryButton, message: "Try again")
// When creating the popup, before its next one-second sample:
replay.publicLabel(errorMessageLabel,
    message: "Unable to update your cart. Please try again.")
// Before reusing this label for customer-specific content:
replay.makePrivateAgain(errorMessageLabel)
// Consent withdrawal callback on the main actor:
replay.revoke()
```

Never pass label.text, names, addresses, emails, order/payment IDs, input values, access tokens or response bodies. Only your supplied public string is sent. Inputs, WebViews, images and unapproved labels remain private; blocked ancestors override child approvals. Strings are limited to 160 Unicode scalars and a 4 KiB encoded per-frame public-text budget. Removing an approval cannot recall previously received content.

iOS 0.3.0: session duration and lifecycle

Keep one recorder through the foreground journey

Do not start/stop on every page or network request. Remove the older five-minute CappedReplay helper if full-session capture is intended. There is no total-session timer in the collector. Up to five frames per part stay small for delivery; a stable session reference and retry-safe batch ID join them into one timeline. An upload interval is not the session duration.

Retain the recorder through ordinary background/foreground transitions on the same approved window. Capture pauses automatically while inactive, hidden, non-key or externally captured, and a partial batch flushes on resigning active. Recheck consent and the key window on return. Missing/background intervals are gaps, not reconstructed activity. Stop before window/account changes or sensitive workflows; restart explicitly and reapply blocks/approvals. Process restart creates a new session.

Revoke consent before flushing

Call setEnabled(false) first on withdrawal. Never normal-stop or flush first. Active frames and queued telemetry are discarded; already sent or in-flight requests cannot be recalled. A later grant requires a new explicit start. Do not create or enable the client while consent is unknown.

Visual and operational limits

This remains a UIKit wireframe, not pixel video. Opaque backgrounds and approved label/button text colors and font size improve recognition. Inputs, image pixels, system alerts, separate windows, keyboards, SwiftUI internals and custom drawing are not captured. Use app-owned UIKit popup labels. Layout is bounded to 128 nodes and 32 hierarchy levels. Fit/100%/150%/200% player zoom aids inspection; it does not increase capture fidelity.

Playback supports at most 10,000 received parts per session. End/start a new session before this limit for exceptionally long journeys. Clock skew, offline queue bounds, quota and process termination can prevent full delivery. Capture duration is not a guarantee that every frame reaches the server.

Shared delivery, retention and troubleshooting

The native queue defaults to 100 records (configurable 1-1000), at most 64 KiB each and retained up to 24 hours. Delivery retries network failures, 408, 429 and 5xx up to five attempts with backoff. A flush callback indicates an attempt, not confirmed storage. An iOS v2 batch keeps its ID for retry idempotency. Suspended or terminated delivery is not guaranteed. App-private queue storage is excluded from backup.

Tenant recording retention defaults to 30 days and is capped by the subscription. Owners/admins manage it in Session Replay. Storage quota spans tenant projects; new uploads are rejected at quota while existing recordings remain. Shortening retention can permanently purge old recordings. Group deletion removes all received parts and rejects late uploads for the session. Public links require expiration and expose only parts received when generated. Expiration, revocation, deletion or retention stops public access.

For short cards, confirm app build/SDK, remove capped helpers and per-screen stop/start, and check iOS session rather than iOS wireframe batch. The published 0.2.0 SDK still sends individual batches. For missing public text, check approved UILabel/UIButton, blocked ancestors, timing and budgets. For delivery failures, inspect DSN/environment, device clock, quota, network errors and queue statistics. Do not remove protected-screen controls to force capture.

Mobile team acceptance checklist

1. Version and installation
Verify the reviewed iOS 0.3.0 tag is published before pinning it; otherwise use candidate source only in staging. Rebuild the app and commit Package.resolved. Android remains 0.2.0.

2. No consent
Cold-launch and decline. No client construction, frames or telemetry. Construct only after the grant.

3. Full journey
Record at least two minutes with navigation, scrolling and quiet foreground time. iOS 0.3.0 should show one reference and multiple parts; Android 0.2.0 remains separate batches.

4. Public popup
Trigger an app-owned UIKit popup with an approved static message. Check readable text, colors and zoom.

5. Private sentinels
Use synthetic names, input values and secure fields. Block a parent containing an approved child. None of that private content may appear in stored frames or playback.

6. Cell reuse
Remove approval before reusing a public label for customer-specific content; verify the next frames contain no private value.

7. Consent revoke
Withdraw while active. Active buffers/queue must purge; no automatic restart. Account for data already sent. Later capture needs a new explicit grant/start.

8. Lifecycle
Check background pause/gap, same-window resumption, window/account change, protected screen and process restart.

9. Playback
Check play/pause, 1x/2x/4x, cross-part seek, Fit/100%/150%/200%, narrow layout and failed-part retry.

10. Offline and duplicates
Disconnect/reconnect; inspect queue bounds. Retry of the same batch must not duplicate session parts or storage usage.

11. Quota and retention
Use isolated synthetic tenants to test quota rejection, retention purge and whole-session/bulk deletion. Verify late-upload rejection and public-link invalidation.

12. External sharing
Share only reviewed synthetic content. Verify mandatory expiration, anonymous access, frozen parts, expiration and revocation.

13. Simulator and device
Run UIKit simulator tests and a physical iPhone/iPad journey. Measure responsiveness, CPU, memory, battery, orientation and capture overhead.

14. Release sign-off
Record app/SDK build, environment/DSN, recording reference, timestamps, device/OS, privacy results and owner approval. Choose production sampling and monitor volume.

Mobile owner: _____ QA owner: _____ App build/date: _____